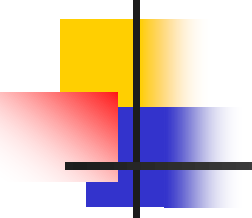


Verilog 硬體描述語言 (I)

- Verilog的時間控制與資料型態
- Verilog的敘述



“wire” Description

□ wire (接線) 的特性，如下：

- 接線是連接硬體元件之連接線。
- 接線必須要被驅動才能改變它的內函值。
- 接線描述的關鍵字為 wire。
- 除非有被宣告成一個向量 (vector)，否則接線是內定為一個位元的值，而且內定值是 “Z”。

□ 範例：

- `wire b, c;` // 宣告2條接線, 且命名為 b 及 c
- `wire a = b & c;` // 宣告1條接線 a, 並指定它為 b & c (驅動)
- `wire d = 1'b0;` // 宣告1條接線 d, 並指定它的值為一個固定值0 (驅動)

“wand (Wired-AND)” Description

❑ “wand” 真值表：

VLSI是用「開集極」(open collector) 元件來達到 wired-AND 的功能。

❑ 範例：wand.V, Wired-AND 敘述

// wand.V: Wired-AND 敘述

```
module wand_test(a, b, c);
```

```
input  a, b;
```

```
output c;
```

```
wand    c;
```

```
    assign c = a;
```

```
    assign c = b;
```

```
endmodule
```

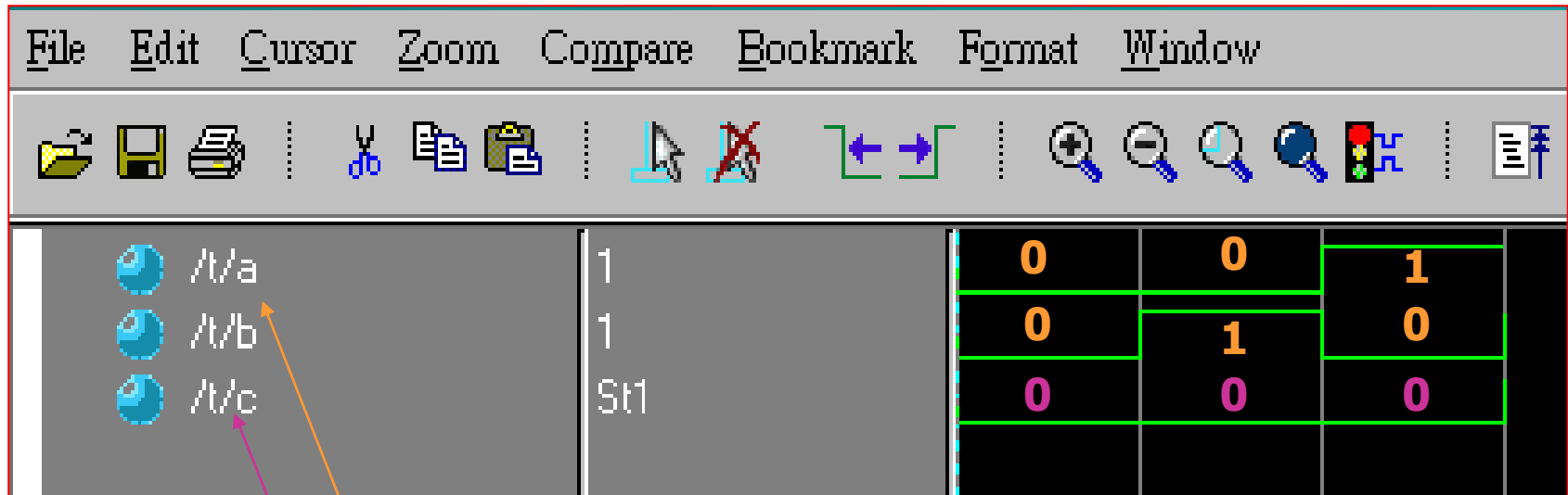
wand					
c		a			
		0	1	X	Z
b	0	0	0	0	0
	1	0	1	X	1
	X	0	X	X	X
	Z	0	1	X	Z

Z= 浮接

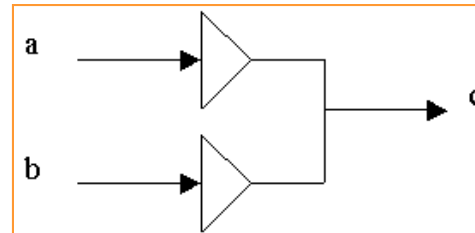
X= 0 or 1

Simulation Waveform

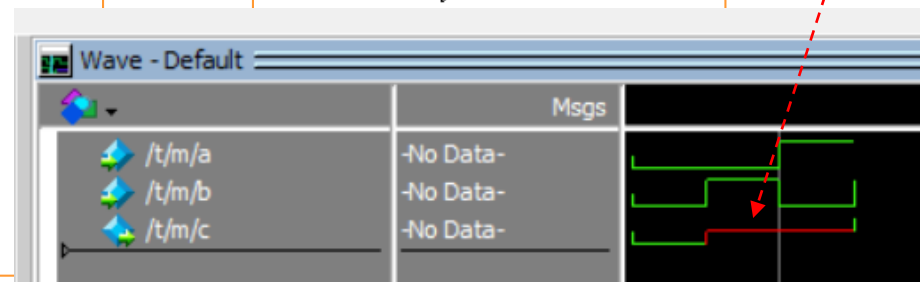
Core:2_03_WAND



```
module wand_test(a, b,  
c);  
input a, b;  
output c;  
wand c;  
    assign c = a;  
    assign c = b;  
endmodule
```



wire c;



“wor (Wired-OR)” Description

□ “wor” 真值表：

VLSI 是用「射極耦合邏輯」(ECL, Emitter Couple Logic) 元件來達到 wired-OR 的功能。

□ 範例：wor.V, Wired-OR敘述

//wor.V, Wired-OR敘述

```
module wor_test(a, b, c);
```

```
input  a, b;
```

```
output c;
```

```
wor  c;
```

```
assign c = a;
```

```
assign c = b;
```

```
endmodule
```

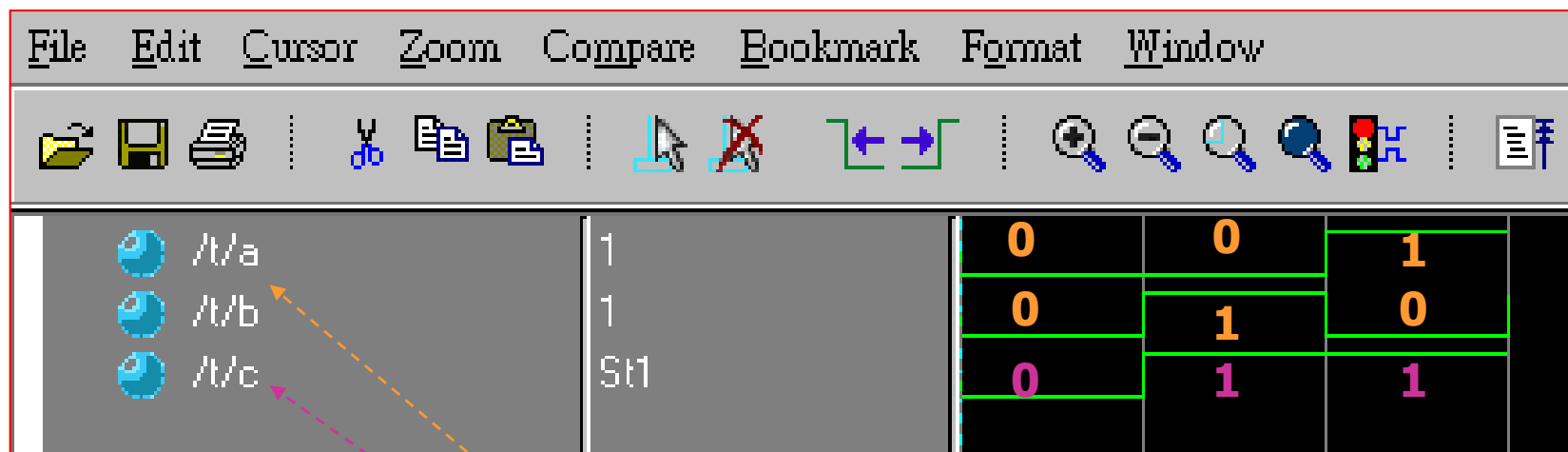
wor					
c		A			
		0	1	X	Z
b	0	0	1	X	0
	1	1	1	1	1
	X	X	1	X	X
	Z	0	1	X	Z

Z= 浮接

X= 0 or 1

Simulation Waveform

Core:2_04_WOR



```
module wor_test(a, b,  
c);  
input  a, b;  
output c;  
wor c;  
    assign c = a;  
    assign c = b;  
endmodule
```

“reg” Description

□ reg (暫存器) 的特性

- 暫存器 (reg) 的功能與變數很像、可以直接給定一個數值，主要的功能在於保持住電路中的某個值；不必像接線 (wire) 要被驅動才能改變它的內函值。
- 暫存器描述的關鍵字為 “reg”。
- 除非有被宣告成一個向量 (vector)、否則reg是內定為一個位元的值，而且內定值是 “X” (Z = don't care, but X = unknown)。

□ 範例：

```
input    [3:0] sum;
output   zero;    // 宣告1個輸出埠, 且命名為 zero
reg      zero;    // 宣告1個暫存器, 且命名為 zero
  always @(sum)
  begin
    if(sum == 0) // 如果 sum = 0
      zero = 1;  // 則, 令zero 一直保持為 1
    else
      zero = 0;  // 否則, 令zero 一直保持為 0
  end
```

How to Use “wire” or “reg” Descriptions

- ❑ “wire” 必須配合 “assign” 來使用，且不能出現在 “always” 的區塊描述裡。

- ❑ 範例：

```
wire a, b, c;  
assign a = b & c;
```

- ❑ “reg” 的使用方法為 “a = b” 格式，也就是必須放在 “always” 的區塊描述裡。

- ❑ 範例：

```
input [3:0] a, b;  
output [3:0] z;  
reg [3:0] z;  
always @(a or b)  
begin  
    z = a + b;  
end
```

wire → assign
reg → always

Vectors Declaration

- ❑ “wire” 和 “reg” 皆可定義為向量(vectors)；若無定義位元長度、則視為被宣告的變數其位元長度為1，也就是一個位元。
- ❑ 向量是一個多位元的元件 (element)。
- ❑ 向量可定義為 **[High#:Low#]** 或 **[Low#:High#]**。
 - 以 [High#:Low#] = [MSB:LSB] 來說，在中括號內之分號左邊的部份為最高位元 (Most Significant Bit, **MSB**)，在中括號內之分號右邊的部份為最低位元 (Least Significant Bit, **LSB**)。
- ❑ 我們可以用向量的表示法來取得向量內之部份訊號。
- ❑ 範例：

```
wire  even;           // 宣告1條接線 even
wire  bus [15:0];      // 宣告1條由16條接線所形成的匯流排, 稱為 bus
reg    bit;           // 宣告1個 1 bit 的 reg
reg    Result [31:0];  // 宣告1個用32個 reg 所形成的 register
```

```
assign even = ~ Result[0]; // 只取 Result 的第0個位元取NOT給even
assign bus[15:0] = Result[15:0]; // 取 Result 的第15至第0個位元給bus
```

Arrays Declaration

- ❑ 陣列的內容可以是：**整數、暫存資料，以及向量**。
- ❑ HDL 編譯器 只能用於描述單維陣列的表示法、不能描述多維陣列。
- ❑ 「陣列」是多個1位元或若干個位元的元件。
- ❑ 語法：

➤ reg [**Bits**, MSB:LSB] Reg_name [Array, High:Low]

(每位陣列的大小)

(陣列的個數)

- ❑ 範例：

➤ reg [15:0] Word; // 1個Register, 這個 Register 為16個 Bits

➤ reg Mem1Bit [1023:0]; // 1024個1 Bits 的 Register

➤ 陣列 : reg [7:0] RegFile [1023:0];

// 1024個Register, 每個 Register 為8個 Bits

Memory Declaration

- ❑ 在 Verilog 中，記憶體像是一個暫存器的陣列，在陣列中的每一個物件就像是一個 Word、每個 Word 可以是一個位元或多個位元。
- ❑ N 個 1 Bit 的暫存器和一個 N Bits 的暫存器是不相同的。
- ❑ 陣列中的註標 (SubScript) 用於標名記憶體中的位址，用於指定我們想要存取某個記憶體元件 (暫存器)。
- ❑ **範例：**

```
reg Bit1, Bit2;                // 二個1bit的暫存空間
reg Memory1Bit [1023:0];       // 1024個1 Bit 的暫存器空間
reg [7:0] Byte1, Byte2;        // 二個8 Bits的暫存空間
reg [7:0] Memory1Byte [1023:0]; // 1024個1 Byte (8 Bits) 的暫存器空間
```

```
Bit1 = Memory1Bit [321];
// 提取位址為 321 的1 Bit 的暫存器給 Bit1
```

```
Memory1Byte [586] = Byte1;
// 將 Byte1 的值存入位址為 586 的Memory1Byte 中
```

```
// 提取在位址為 125 的Memory1Byte 中之內容(8位元) 的第4個位元給 Bit2，
// 必需藉由二次提取的方式才可以完成
```

```
Byte2 = Memory1Byte [125]; // 首先：提取位址為Memory1Byte [125]
Bit2 = Byte2 [4]; // 然後：提出第4個位元
```

Bit-Selects and Part-Selects

❑ **位元選擇 (Bit-Selects)** 與 **部份選擇 (Part-Selects)** 是指選擇某個由向量 (Vectors) 表示法或者是陣列 (Arrays) 表示法的變數之某些特定的部份。

❑ **位元選擇 (Bit-Selects)**

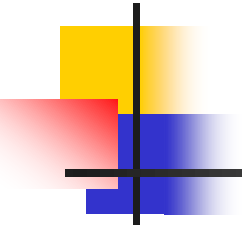
位元選擇是選擇來自於 1條 wire (接線)、1個 register (暫存器) 或 parameter vector (參數向量) 的單一位元。

➤ 範例：
wire [7:0] a, b, c;
assign c[0] = a[0] & b[0]; // 選擇 a[0] 和 b[0] 作 & 後, 給 c[0]

❑ **部份選擇 (Part-Selects)**

部份選擇是選擇來自於 wire (接線)、register (暫存器) 或 parameter vector (參數向量) 的一群位元。

➤ 範例：
wire [7:0] a, b, c;
assign c[7:0] = a[7:0] & b[7:0];
// 選擇 a[7:0] 和 b[7:0] 作 & 後, 給 c[7:0]



Verilog 硬體描述語言 (I)

- Verilog的時間控制與資料型態
- Verilog的敘述

“Input”/“Output” Descriptions

- ❑ Verilog 的輸出入埠敘述，包括有：輸入埠 (input)、輸出埠 (output)、雙向埠 (inout) 等三種敘述。

- ❑ **input (輸入埠) 敘述**

input (輸入埠): 所定義的埠，具有輸入埠的特性。

- ❑ **output (輸出埠) 敘述**

output (輸出埠): 所定義的埠，具有輸出埠的特性。

- ❑ 範例：input (輸入埠) 及 output (輸出埠) 敘述的 Verilog 片段範例

```
input  [3:0] A, B; // 宣告二個具有4個位元的輸入埠 A 及 B
```

```
output [3:0] Z;    // 宣告一個具有4個位元的輸出埠 Z
```

```
reg    [3:0] Z;    // 宣告一個具有4個位元的暫存器型態變數 Z
```

```
always @(A or B) //當輸入的A或B值與預定值有變動時，即執行
```

```
begin
```

```
    Z = A + B;
```

```
end
```

- 



Verilog Generic Description

- ❑ Verilog 常用 (Generic) 的敘述有：parameter(常數)、assign(指定)、always(當)、if(如果)、if ... else ... (如果.../否則...)、case (當符合)、casex(含x的case)、casez(含z的case)、for(當...重覆執行)、function(函數或副程式)、task (工作或副程式)...等敘述。
- ❑ 說明 if 與 case 這二大類敘述的使用時機。
- ❑ function 與 task 敘述的差異比較。

“parameter” Description

❑ 在模組中、使用**關鍵字parameter**來定義一個**常數**，對於可擴性設計的模組是非常好用的。

❑ 範例：

ScalableDesign 模組的功能為處理2個輸入資料 a 與 b 作16個位元的 & 動作後，將結果給 c。在第3行中有 parameter 這個敘述，且設定了一個名為 width 的常數值為16；在**第4行及第5行**中分別引用了input [width-1:0] a, b; output [width-1:0] c; 相當於 input [15:0] a, b; output [15:0] c; 我們只要把在第3行中 parameter 這個敘述所指定之常數 width 的值改成所需要的為位元組長度即可達到易於修改的目的。

// EasyScal.V，易於修改的模組設計，使用 parameter 敘述

```
module ScalableDesign(a, b, c);
```

```
parameter width = 16;
```

```
input [width-1:0] a, b;
```

```
output [width-1:0] c;
```

```
wire [width-1:0] c;
```

```
assign c = a & b;
```

```
endmodule
```



“assign” Description

□ assign 敘述的功能如下：

- 驅動某個值到 **wire, wand, wor 或 tri**。
- 用於**資料處理模型 (Data Flow Model)** 的描述。
- 用於**描述組合邏輯 (Combinational circuits)** 電路。

□ 範例：

wire a, b, c; // 宣告三個接線型態的變數, 分別為: a, b 及 c

assign a = b & c; // 指定 a = b 和 c 作 & 運算

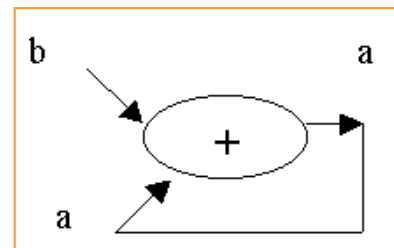
wire b, c; // 宣告二個接線型態的變數, 分別為: a 及 b

wire a = b & c; // 宣告並且指定 a = b 和 c 作 & 運算

□ assign 敘述的問題如下：

- 我們必須避免使用「迴路」(logic loop)。
- 例：assign a = b + a;

這種寫法在等號 (=) 的左右二邊都出現相同的變數，HDL 編譯及電路合成過程中會自動拆解這種寫法，而且合成出來的電路在電路的模擬擬時會發生時序上的錯誤，出現不預期的結果出現。



"always" Description (1)

□ **always** 敘述的觀念有點像是一名警衛一樣，隨時在監督外界(輸出入埠)訊號的情況。當外界的訊號一有變化隨即告知模組內部要配合作相對應的處理。

□ 語法：

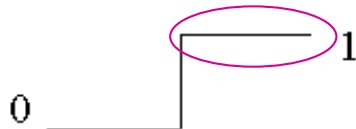
```
always @( event-expression [or event-expression*] )  
begin  
    <statement_or_null>  
end
```

□ Case I：在event-expression (事件表示式)中的變數，必須在 always @ 的begin ... end 這個 block 中出現。Synopsys 將會使用 Latches 來合成出 Gate Level 電路。

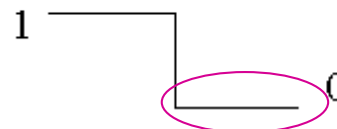
➤ 範例：準位觸發 (Level Trigger)

```
always @(a or b or c) // 只要訊號 a, b 或 c 有變化 (0→1 或 1→0)  
begin  
    f = a & b & c; // 則更新 f 的值, f = a & b & c  
end
```

訊號由 0 轉變為 1 (0 → 1)



訊號由 1 轉變為 0 (1 → 0)



"always" Description (2)

- Case II：在 event-expression 中訊號名稱的前面，如果加上 **posedge edge trigger** (正緣觸發) 或者是 **negedge edge trigger** (負緣觸發) 這二個識別字 (Keyword) 的其中一種，則 Synopsys 將會使用 flip-flop 來合成出 Gate Level 電路。

➤ 範例1：posedge 正緣觸發

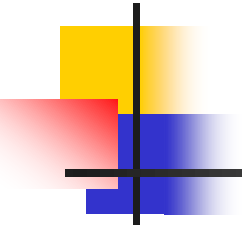
```
always @(posedge Clock) // 只要訊號 Clock 為正緣觸發
begin
    f = a & b & c;    // 則更新 f 的值, f = a & b & c
end
```

訊號 Clock 為 negedge edge trigger (負緣觸發)



➤ 範例2：negedge 負緣觸發

```
always @(negedge Clock) // 只要訊號 Clock 為負緣觸發
begin
    f = a & b & c; // 則更新 f 的值, f = a & b & c
end
```



"if" Description (1)

- if 敘述：為一種二路分支選擇的條件式判斷敘述，通常是用來作為判斷訊號的值之後、再選擇要作的處理。
- If的語法規定(表示法)：
if (<expression>)
 <statement_or_null>
或
if (<expression>)
 begin
 <statement_or_null>
 end
- if 敘述能處理「正準位」觸發 (Positive Level Trigger) 以及「負準位」觸發 (Negative Level Trigger) 等二種訊號。

正準位觸發 ≠ 正緣觸發
負準位觸發 ≠ 負緣觸發

"if" Description (2)

□ 正準位觸發 (Positive Level Trigger) :

只要訊號的值是「正準位」，則會作相對應的處理。

□ 範例1：正準位觸發 (Positive Level Trigger) 的第一種寫法

```
always @(posedge Clock) //正緣觸發
```

```
begin
```

```
    if (Reset == 1'b1) // 如果 Reset 的值等於 1 (正準位觸發)
```

```
        Out = 0; // 則令 Out = 0
```

```
    else
```

```
        Out = In_Data;
```

```
end
```

□ 範例2：正準位觸發 (Positive Level Trigger) 的第二種寫法

```
always @(posedge Clock) //正緣觸發
```

```
begin
```

```
    if (Reset) // 如果 Reset 的值等於 1 (正準位觸發)
```

```
        Out = 0; // 則令 Out = 0
```

```
    else
```

```
        Out = In_Data;
```

```
end
```

"if" Description (3)

❑ 負準位觸發 (Negative Level Trigger)

只要訊號的值是「負準位」，則會作相對應的處理。

❑ 範例3：負準位觸發 (Negative Level Trigger) 的第一種寫法

```
always @(posedge Clock) //正緣觸發
```

```
begin
```

```
    if (Reset == 1'b0)           // 如果 Reset 的值等於 0 (負準位觸發)
```

```
        Out = 0;                 // 則令 Out = 0
```

```
    else
```

```
        Out = In_Data;
```

```
end
```

❑ 範例4：負準位觸發 (Negative Level Trigger) 的第二種寫法

```
always @(posedge Clock) //正緣觸發
```

```
begin
```

```
    if (!Reset)                 // 如果 Reset 的值等於 0 (負準位觸發)
```

```
        Out = 0;                 // 則令 Out = 0
```

```
    else
```

```
        Out = In_Data;
```

```
end
```

"if...else" Description (1)

❑ “if ... else ...” 敘述：為一種二路分支選擇的條件式判斷敘述，其意思是：如果... / 否則 ... 敘述。

❑ BNF 表示法：

➤ 第一種表示法：

```
if (<expression> )  
    <statement_or_null>  
else  
    <statement_or_null>
```

BNF=Binary Normal Format
=Backus-Naur Form (人名)

➤ 第二種表示法：

```
if (<expression> )  
begin  
    <statement_or_null>  
end  
else  
begin  
    <statement_or_null>  
end
```


"if...else" Description (2)

- ❑ 實例：底下的二個範例功能是相同的，只是描述的語法不同。
- ❑ **if 電路合成**：Synopsys 將會使用 Priority Decoder 來合成出 Gate Level 電路。

- ❑ **範例1：**

```
always @(X or Y or Z)
begin
    if (Z)                                // 優先權 (Priority) 最高
        Out = Result4;
    else
        if (Y)                            // 優先權 (Priority) 第2高
            Out = Result3;
        else
            if (X)                         // 優先權 (Priority) 第3高
                Out = Result2;
            else
                Out = Result1;            // 優先權 (Priority) 最低
end
```

"if...else" Description (3)

電路合成時的順序，而模擬非如此。

□ 範例2：

```
always @(X or Y or Z)
begin
```

```
    Out = Result1;
```

```
    if (X)
```

```
        Out = Result2;
```

```
    if (Y)
```

```
        Out = Result3;
```

```
    if (Z)
```

```
        Out = Result4;
```

```
end
```

// 優先權 (Priority) 最低

// 優先權 (Priority) 第3高

// 優先權 (Priority) 第2高

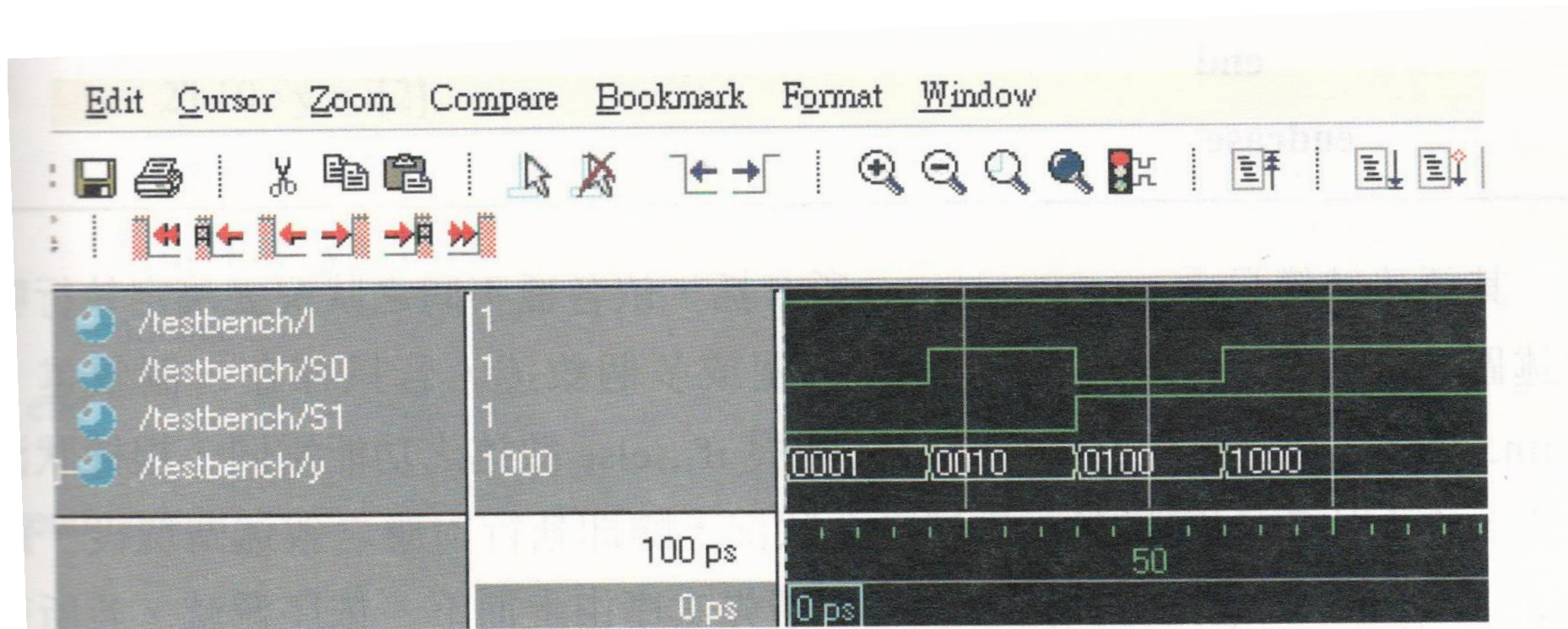
// 優先權 (Priority) 最高

PUSH-PULL Concepts

1-4解多工器

(5-001)

1對4解多工器的模擬結果



//1 to 4 demultiplexer using nesting if...else..statement

// Filename: demul1_4_if.v

module demul1_4_if(y, I, S0, S1);

output [3:0]y;

input I;

input S0, S1;

reg [3:0]y;

always@(I or S0 or S1)

begin

if(S1 == 1'b0)

begin

if(S0 == 1'b0)

y = {3'b000, I}; //S=(00)

else

y = {2'b00, I, 1'b0}; //S=(01)

end

else

begin

if(S0 == 1'b0)

y = {1'b0, I, 2'b0}; //S=(10)

else

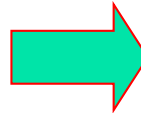
y = {I, 3'b0}; //S=(11)

end

end

endmodule

Test bench



//Filename:comp2_g_tb.v

module demul1_4_if_tb();

wire [3:0]y;

wire I;

wire S0, S1;

demul1_4_if uut(.y(y), .I(I), .S0(S0), .S1(S1));

reg clk;

reg [2:0]data;

initial

begin

clk = 0;

data = 3'b0;

end

always

begin

#10 clk = ~clk;

end

always@(posedge clk)

begin

if(data < 3'b111)

data = data+1;

else

data = 3'b0;

end

assign I = data[2];

assign S1 = data[1];

assign S0 = data[0];

endmodule

“case” Description (1)

□ case 敘述(Parallel decoder):

- 為一種多路分支選擇的敘述。
- 如果電路中所有可能的分支判別條件都被指定，稱為 full case。

□ BNF 主要表示法：

case (expr)

```
case_item1 : begin
    <statement_or_null>
end
case_item2 : begin
    <statement_or_null>
end
...
default : begin
    <statement_or_null>
end
endcase
```

BNF=Binary Normal Format
=Backus-Naur Form (人名)

“case” Description (2)

- 以下的範例中，其功能是相同的，只是描述的語法不同，且 Synopsys 將會使用 **Priority Decoder** 來合成出 Gate Level 電路。

- **範例1(case) :**

```
always @(u or v or w)
```

```
begin
```

```
case (2'b11) // 一個狀態(11)內有三個不同的變數(u,v,w)，故有優先權
```

```
  u      : Out = 3'b001; // 優先權 (Priority) 最高
```

```
  v      : Out = 3'b010; // 優先權 (Priority) 次高
```

```
  w      : Out = 3'b100; // 優先權 (Priority) 最低
```

```
  default : Out = 3'b000;
```

```
endcase
```

```
end
```

- **範例2(if...else...) :**

```
always @(u or v or w)
```

```
begin
```

```
  if u = 2'b11 // 優先權 (Priority) 最高
```

```
    Out = 3'b001;
```

```
  else
```

```
    if v = 2'b11 // 優先權 (Priority) 第2高
```

```
      Out = 3'b010;
```

```
    else
```

```
      if w = 2'b11 // 優先權 (Priority) 最低
```

```
        Out = 3'b100;
```

```
      else
```

```
        Out = 3'b000;
```

```
end
```

2'b11為已知的狀態，其下有三個變數(u, v, w)，其優先權各不同。

“case” Description (3)

- ❑ 我們可以指定電路為 **parallel case** 的特性，這樣 Synopsys 就不會使用 Priority Decoder 來合成出 Gate Level 電路，而會產生 Parallel Decoder 電路。

- ❑ **範例3：**

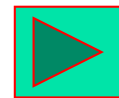
```
always @(select)
begin // synopsys parallel_case
  case (select) // 三個狀態(00, 01, 11)供選擇,優先權相同
    2'b00 : Out = in1;
    2'b01 : Out = in2;
    2'b11 : Out = in4;
  endcase
end
```

Select與temp均為變數，以下所對應的是各種狀態，其優先權均相同。

- ❑ **範例4(特例)：**

```
always @(X or Y or Z)
begin
  temp = {X, Y, Z}; // X=1, Y=1, Z=1 會出錯，需要修正如紅色所述
  casex (temp) // synopsys parallel_case, 三個狀態(1xx, x1x, xx1)供選擇, 優先權不同.
    3'b1xx: Value = Result1; // X = 1, 優先權 (Priority) 最高
    3'bx1x: Value = Result2; // Y = 1, 優先權 (Priority) 次高
    3'bxx1: Value = Result3; // Z = 1, 優先權 (Priority) 最低
    default: Value = Result4; // others
  endcase
end
```

x → 0,1



“casex” Descriptions (1)

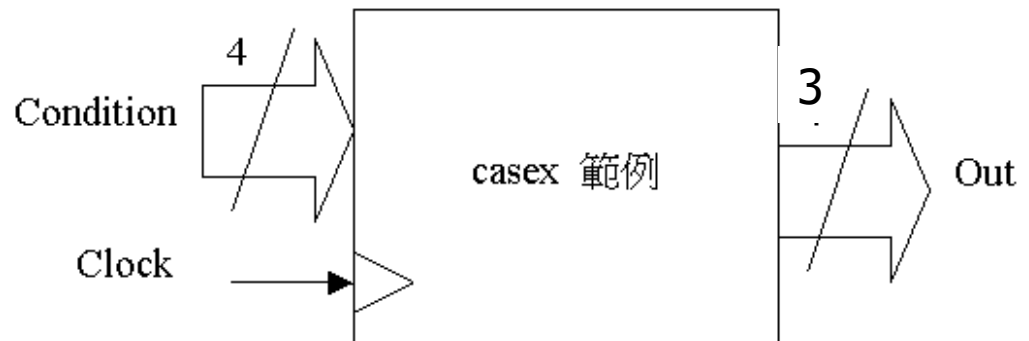
□ casex 敘述

- 允許 case_item 的值為「？」，在“測試資料碼”時，「？」可以用 0 或 1 來當作測試的值。
- 不允許 expr 中有 ?、z 或 x。

X = 0 or 1

□ BNF 表示法：

```
casex (expr)
  case_item1 : begin
    <statement_or_null>
  end
  case_item2 : begin
    <statement_or_null>
  end
  ...
  default : begin
    <statement_or_null>
  end
endcase
```



“casez” Description (2)

□ “casez” 敘述

Z = 0, 1, ? or z

允許 case_item 的值為「?」，在“測試資料碼”時，「?」可以用 0、1、? 或 z 來當作測試的值，也就是任意值均可。

□ 範例:

// casez.v : casez 敘述範例, z 為高阻抗 (High Impedance)

// 我們希望時脈訊號 (Clock) 有正緣觸發時,才判斷Condition 值,以決定輸出值 Out

```
module casez_machine (Clock, Condition, Out);
```

```
input    Clock;
```

```
input    [1:0] Condition;
```

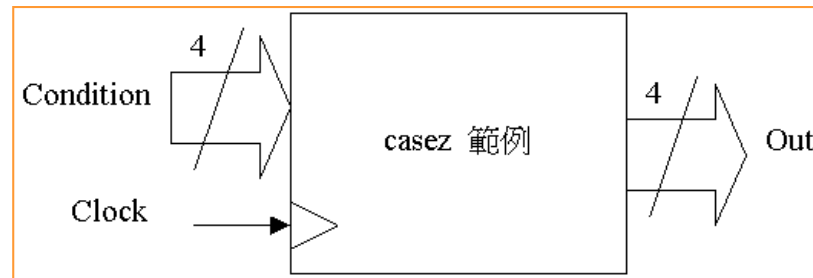
```
output   [2:0] Out;
```

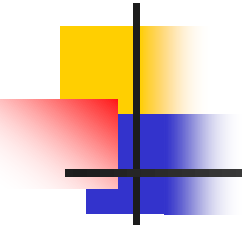
```
reg      [2:0] Out;
```

```
always @(posedge Clock) // 當時脈訊號有正緣觸發時
```

```
begin
```

```
    casez (Condition)
```





“casez” Description (3)

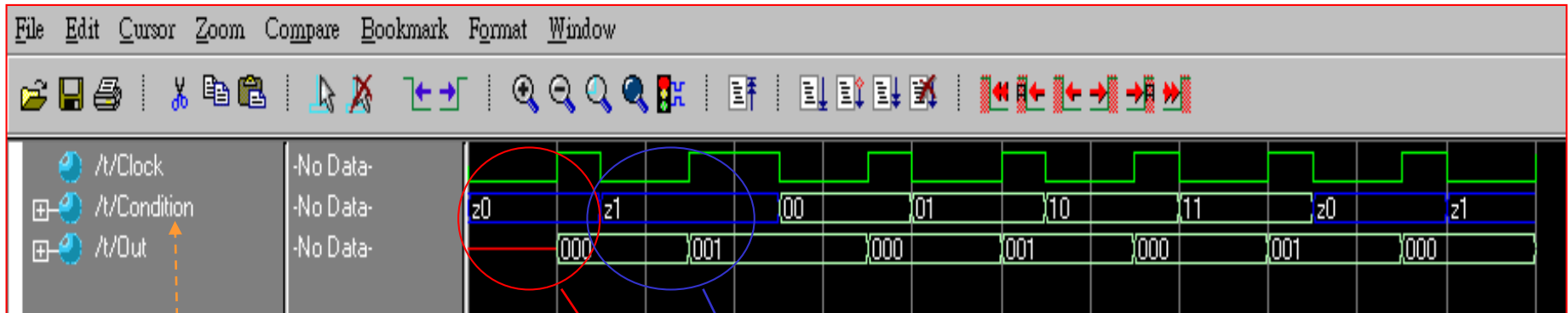
casez (Condition)

```
2'bz0:    // 只要 Condition[0] = 0, 則 Out = 3'b000;
    begin
        Out = 3'b000;
    end
2'bz1:    // 只要 Condition[0] = 1, 則 Out = 3'b001;
    begin
        Out = 3'b001;
    end
default : // Condition 為其他值, 則 Out = 3'b111;
    begin
        Out = 3'b111;
    end
endcase
end
endmodule
```

下列為誤:
casez(Condition)
2'bx0:
2'bx1:
default:

Simulation Waveform

Core:2_07_CASEZ



只要 Condition[0] = 0

Out = 3'b000

只要 Condition[0] = 1
Out = 3'b001

```
2'bz0:    // 只要 Condition[0] = 0, 則 Out = 3'b000;  
2'bz1:    // 只要 Condition[0] = 1, 則 Out = 3'b001;  
default : // Condition 為其他值, 則 Out = 3'b111;  
          (實際上無其他值存在, 除非考量到 X 或 Z)
```

It is correct to replace casez with casex

在“測試資料碼”時，「？」可以用 0、1、?或 z 來當作測試的值，也就是任意值均可。

2's complementary of 8-bits (1)

```
// TwosComp.V : 8個Bits的2的補數  
// 範例 : In_Data = 1101_0011, 1101_0010, 0101_0000  
//      XOR ^1111_1110 (fe), ^1111_1100, ^1110_0000  
// 運算結果 = 0010_1101, = 0010_1110, =1011_0000  
// ➔ 等於 2's = 0010_1101, = 0010_1110, =1011_0000
```

```
module TwosComp (In_Data, Out_Data);
```

```
input  [7:0] In_Data;
```

```
output [7:0] Out_Data;
```

```
reg    [7:0] Out_Data;
```

```
always @(In_Data)
```

```
begin
```

```
    casex (In_Data) //synopsys parallel_case
```

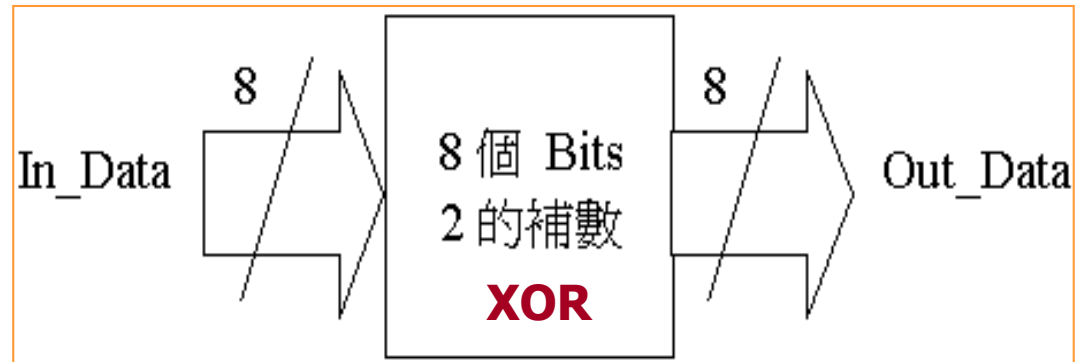
```
        8'b???????1 : Out_Data = In_Data ^ 8'hfe; // fe=1111_1110
```

case~~x~~→ casez: O.K.(V)

case~~x~~→ case: Default (X)

Case~~x~~→ 8'bxxxxxxxx1: O.K.(V)

casez→ 8'bxxxxxxxx1 : Default (X)



2's complementary of 8-bits (2)

```
8'b?????10: Out_Data = In_Data ^ 8'hfc; // fc=1111_1100
```

```
8'b?????100: Out_Data = In_Data ^ 8'hf8; // f8=1111_1000
```

```
8'b????1000: Out_Data = In_Data ^ 8'hf0; // f0=1111_0000
```

```
8'b???10000: Out_Data = In_Data ^ 8'he0; // e0=1110_0000
```

```
8'b??100000: Out_Data = In_Data ^ 8'hc0; // c0=1100_0000
```

```
8'b?1000000: Out_Data = In_Data ^ 8'h80; // 80=1000_0000
```

```
8'b10000000: Out_Data = In_Data ^ 8'h00; // 00=0000_0000
```

```
default: Out_Data = In_Data; // In_Data = 0 (0的2's, 還是0)
```

```
endcase
```

```
end
```

```
endmodule
```

XOR Function: 相同為0，不同為1。

In_Data = 1101_0011

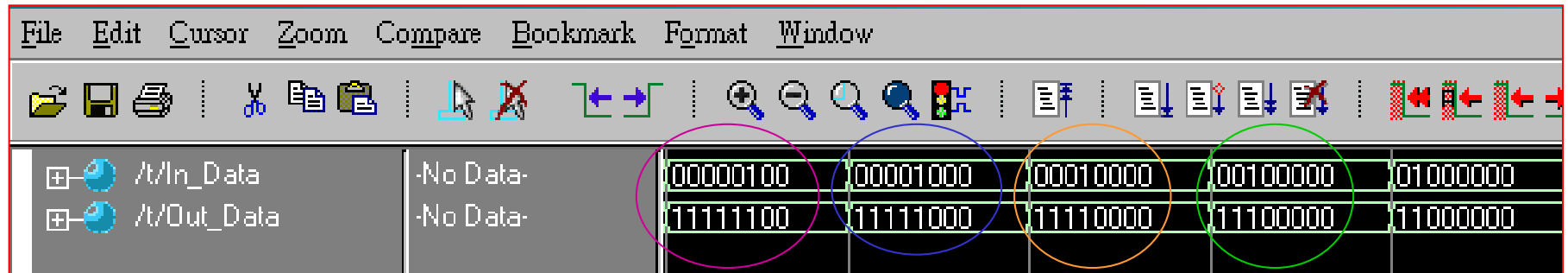
XOR ^1111_1110 (fe)

運算結果 = 0010_1101 (**2's=1's+1**)

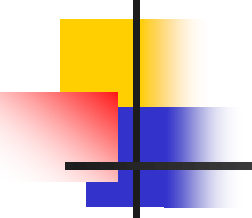
Simulation Waveform

2's complement = 1's
complement + 1

Core:2_09_TWO_COMPL

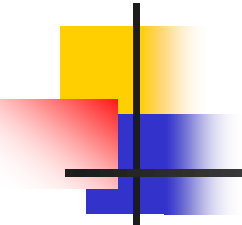


```
8'b???????1: Out_Data = In_Data ^ 8'hfe; // fe=1111_1110
8'b???????10: Out_Data = In_Data ^ 8'hfc; // fc=1111_1100
8'b???????100: Out_Data = In_Data ^ 8'hf8; // f8=1111_1000
8'b?????1000: Out_Data = In_Data ^ 8'hf0; // f0=1111_0000
8'b???10000: Out_Data = In_Data ^ 8'he0; // e0=1110_0000
8'b??100000: Out_Data = In_Data ^ 8'hc0; // c0=1100_0000
8'b?1000000: Out_Data = In_Data ^ 8'h80; // 80=1000_0000
8'b10000000: Out_Data = In_Data ^ 8'h00; // 00=0000_0000
default: Out_Data = In_Data;
```



How to Select “if” or “case”

- 對於 “if” 與 “case” 這二大類敘述的使用時機，通常會作下列的考慮：
- 如果 “if ... else ...” 形成的描述太長了，則改用 “case” 敘述會比較清楚易懂。但並不是每個用 “if ... else ...” 形成的描述都能改用 “case” 敘述來描述的。
 - 如果您知道您在 “case” 敘述中所描述的指定動作具有互斥 (mutual exclusive) 的特性時，請使用 // synopsys parallel_case，用以讓 Synopsys 使用 **Parallel Decoder** 來合成出 Gate Level 電路。
 - 互斥 (mutual exclusive) 的特性是指所有可能的分支判別情形均已放入各個 case 敘述的選項之中。
 - 如果您沒有將所有可能的分支判別條件都指定，但您知道其他您所未指定的條件結果都不可能發生，則您能使用 // synopsys full_case 將您的電路指定為 full case 的特性，也就是自動將沒用的情況為忽略 (don't care). 用以防止 Synopsys 使用 Latch 來合成出 Gate Level 電路。



"for" Description (1)

□ **“for” 敘述**主要是提供一個簡單的方法來描述具有重覆特性的敘述。

□ BNF 表示法：

```
for (<assignment> ; <expression> ; <increment or decrement> )
```

```
begin
```

```
    <statement>
```

```
end
```

或

```
for (index = low_range; index < high_range; index = index + step)
```

或

```
for (index = high_range; index > low_range; index = index - step)
```

或

```
for (index = low_range; index <= high_range; index = index + step)
```

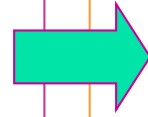
或

```
for (index = high_range; index >= low_range; index = index - step)
```

"for" Description (2)

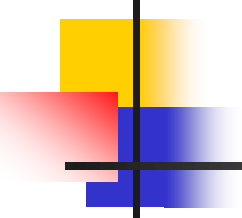
□ 範例：

```
always @(a or b)
begin. for_loop_test
  integer i;
  for (i = 0; i <= 3; i = i+1)
  begin
    Out[i] = a[i] ^ b[i];
    c = (a[i] | b[i]) & c;
  end
end
```



□ 其中 **for loops** 內的敘述在編譯 (Compile Time) 時會被展開成下列的等效敘述。

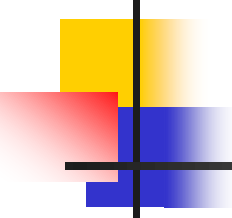
```
always @(a or b)
begin
  Out[0] = a[0] ^ b[0];
  Out[1] = a[1] ^ b[1];
  Out[2] = a[2] ^ b[2];
  Out[3] = a[3] ^ b[3];
  c = (a[0] | b[0]) & (a[1] | b[1]) &
      (a[2] | b[2]) & (a[3] | b[3]) & c;
end
```



範例練習

8位元二進制碼轉格雷碼

(5-003)



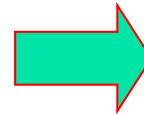
狀態名稱	二進制碼	格雷碼	One-Hot 碼	(5, 2)-碼 (K-Hot 碼)
	編碼所需的 正反器數目 3 個	編碼所需的 正反器數目 3 個	編碼所需的 正反器數目 8 個	編碼所需的 正反器數目 5 個
S1	000	000	00000001	00011
S2	001	001	00000010	00101
S3	010	011	00000100	01001
S4	011	010	00001000	10001
S5	100	110	00010000	00110
S6	101	111	00100000	01010
S7	110	101	01000000	10010
S8	111	100	10000000	01100

```
//convert 8 bit binary code to gary code
//Filename: bin2gra.v
//
module bin2gra(Gry, Bin);
    parameter length = 8;
    output [length-1:0]Gry;
    input [length-1:0]Bin;

    reg [length-1:0]Gry;
    integer i;

    always@(Bin)
        begin
            for(i = 0; i < length - 1; i = i + 1)
                begin
                Gry[i] = Bin[i] ^ Bin[i+1];
                end
                Gry[i] = Bin[i];
            end
        endmodule
```

Test bench



```
module bin2gra_tb();
    parameter length = 8;
    wire [length-1:0]Gry;
    reg [length-1:0]Bin;

    reg clk;

    bin2gra bin2gra(.Gry(Gry),
                    .Bin(Bin)
                    );

    initial
        begin
            Bin = 0;
            clk = 0;
        end

    always
        #5 clk = !clk;

    always@(posedge clk)
        Bin = Bin + 1;

endmodule
```

While 與 forever 迴圈

1041029

➤ 語法格式(while):

```
while (條件判斷表示式)
begin
    <敘述>
end
```

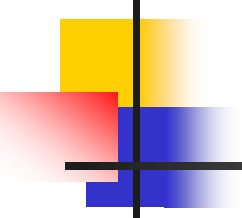
➤ 語法格式(forever):

```
forever
begin
    <敘述>
end
```

➤ 範例:

產生週期為40個時間單位之時脈信號，且於500個時間單位時停止模擬動作。

```
parameter duty_cycle=20;
parameter end_time=500;
initial
begin:clock_sequence
    clock=0;
    forever
        #duty_cycle clock=~clock;
    end
initial
    #end_time disable clock_sequence;
```

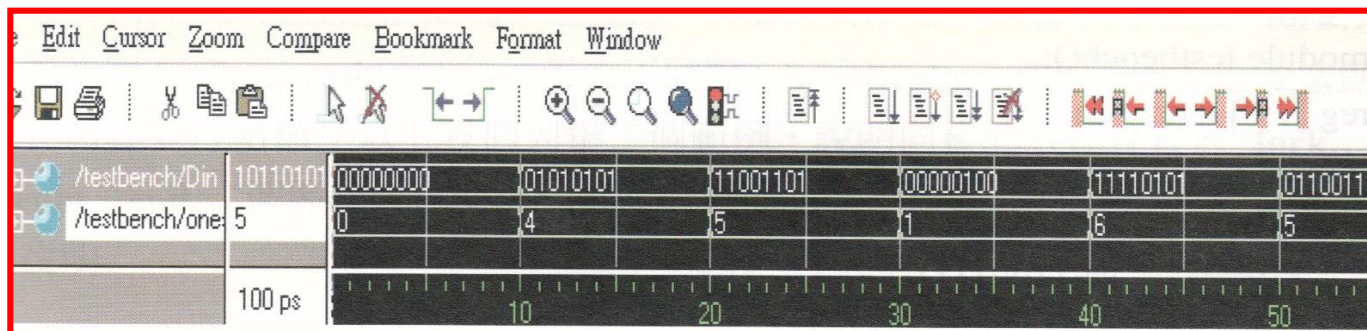


範例練習

repeat迴圈完成位元組中
位元為1的計數
(5-004)

8位元資料計數器的模擬結果

#0	ones = 0,	Din = 00000000
#10	ones = 4,	Din = 01010101
#20	ones = 5,	Din = 11001101
#30	ones = 1,	Din = 00000100
#40	ones = 6,	Din = 11110101
#50	ones = 5,	Din = 01100111
#60	ones = 2,	Din = 10000010
#70	ones = 4,	Din = 00111100
#80	ones = 5,	Din = 10110101



➤ 範例:

讓迴圈執行8次，每次資料tmpr右移(>>)一位，並判定其LSB tmpr[0]是否為1，以計數出現1的次數，直至迴圈執行8次為止。

//count one bit in a 8 bit word by using repeat loop

//Filename: repeat_1s.v

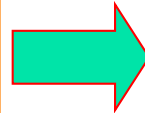
```
module repeat_1s(ones, Din);
    output [3:0]ones;
    parameter length = 8;
    input [length-1:0]Din;

    reg [length-1:0]tmpr;
    reg [3:0]ones;
    reg [3:0]Cout;

    always@(Din)
        begin
            Cout = 4'b0;
            tmpr = Din;

            repeat(length)
                begin
                    if(tmpr[0])
                        Cout = Cout + 4'b0001;
                    tmpr = tmpr >> 1;
                end
            ones = Cout;
        end
endmodule
```

Test bench



```
module repeat_1s_tb();
    wire [3:0]ones;
    parameter length = 8;
    reg [length-1:0]Din;

    repeat_1s repeat_1s(.ones(ones), .Din(Din) );

    reg clk;

    initial
        begin
            clk = 0;
            Din = 0;
        end

    always
        #10 clk = !clk;

    always@(posedge clk)
        begin
            if(Din < 8'b11111111)
                Din = Din + 1;
            else
                Din = 0;
        end
endmodule
```

BCD碼（Binary-Coded Decimal）用4位二進制數來表示十進制數中的0~9這10個數碼。4位二進制正常情況下是在值為15之後產生進位，但如果是BCD碼加法器，那麼為了實現4位二進制在值為9之後就要產生進位，那麼就可以在值大於9的時候，在該值的基礎上加上6，使其自動產生進位。因為加上6之後，此時的4位二進制的值剛好是15，即1111，所以此時就產生進位。二、verilog實現

範例練習

四位元BCD加法器

(5-005)

實驗四：BCD (Binary-Coded Decimal)加法器

「BCD 加法器」是直接對兩個二進位數做加法運算，其結果直接以十進位數來表示。因4位元的二進位在值為15之後會產生進位現象，而BCD碼在值為9之後會產生進位現象。

運算式推演

$$\begin{array}{rcccc}
 & A_3 & A_2 & A_1 & A_0 \\
 + & B_3 & B_2 & B_1 & B_0 \\
 \hline
 C_4 & S_3 & S_2 & S_1 & S_0
 \end{array}$$

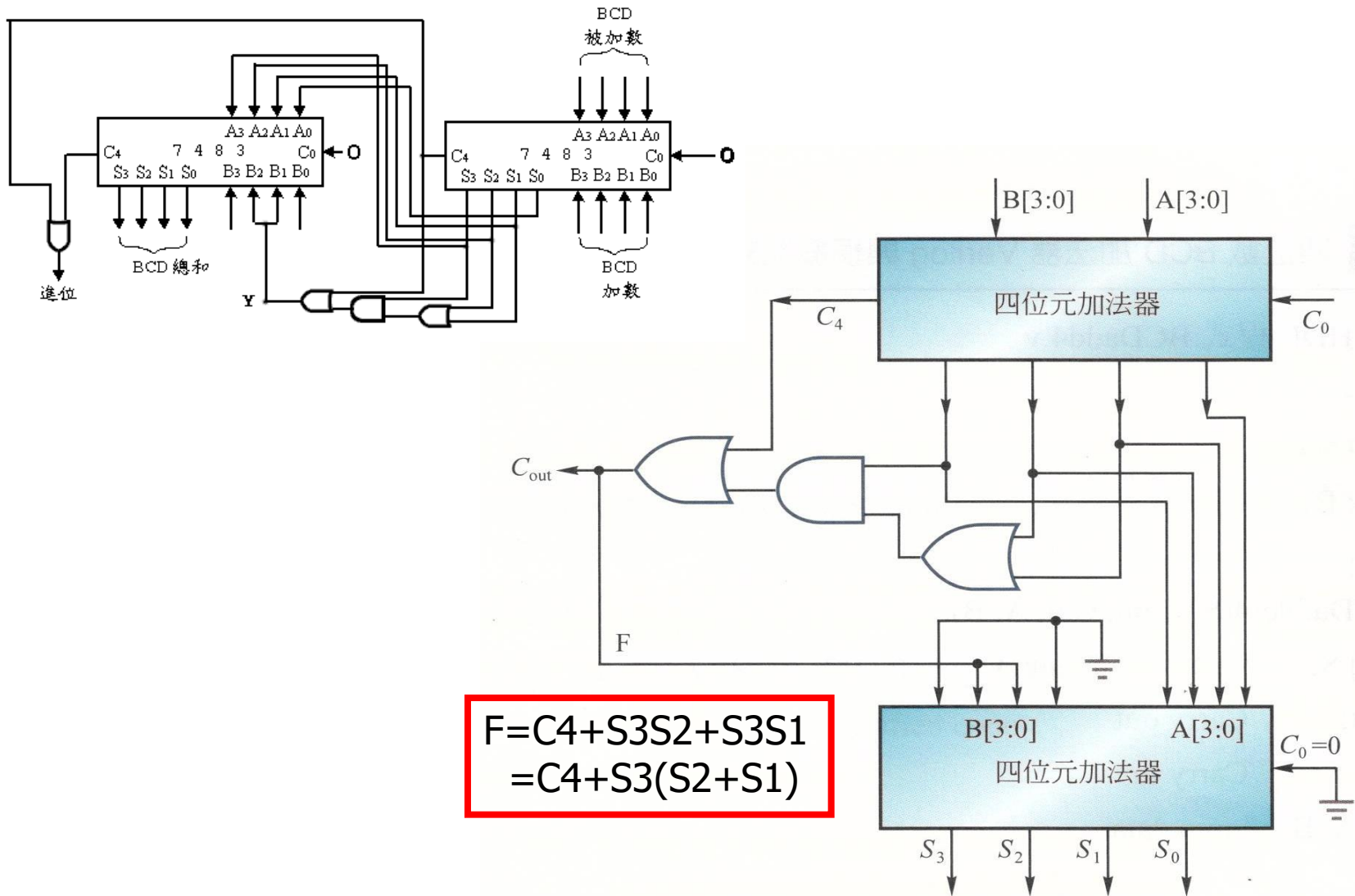
C4	S3	S2	S1	S0	BCD
0	1	0	1	0	(10)
0	1	0	1	1	(11)
0	1	1	0	0	(12)
0	1	1	0	1	(13)
0	1	1	1	0	(14)
0	1	1	1	1	(15)
1	0	0	0	0	(16)
1	0	0	0	1	(17)
1	0	0	1	0	(18)

		S ₁ S ₀			
		00	01	11	10
S ₃ S ₂	00	0	0	0	0
	01	0	0	0	0
	11	1	1	1	1
	10	0	0	1	1



$$\begin{aligned}
 F &= C_4 + S_3S_2 + S_3S_1 \\
 &= C_4 + S_3(S_2 + S_1)
 \end{aligned}$$

四位元BCD加法器電路方塊圖



//4 bit BCD adder

//ex6.21

module BCDadder4(S, Cout, Cin, A, B);

output [3:0]S;

output Cout;

input Cin;

input [3:0]A, B;

wire [3:0]S_tmp;

wire C4;

reg [3:0]B_mod;

reg F;

需配合程式說明

adder4 BINADD(.A(A),
 .B(B),
 .Cin(Cin),
 .S(S_tmp),
 .Cout(C4)
);

adder4 MODADD(.A(S_tmp),
 .B(B_mod),
 .Cin(1'b0),
 .S(S),
 .Cout()
);

always@(Cin or A or B or C4 or S_tmp)

begin

F = (C4 | (S_tmp[3] & (S_tmp[2] | S_tmp[1])));

B_mod = {1'b0, F, F, 1'b0};

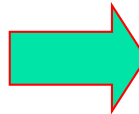
end

assign Cout = F;

$$F = C4 + S3S2 + S3S1 \\ = C4 + S3(S2 + S1)$$

endmodule

Test bench



module BCDadder4_tb();

wire [3:0]S;

wire Cout;

wire Cin;//input

wire [3:0]A, B;//input

reg [8:0]data;

reg clk;

BCDadder4 BCDadder4(.S(S),
 .Cout(Cout),
 .Cin(Cin),
 .A(A),
 .B(B)
);

initial

begin

#0 clk = 0;

data = 0;

end

always

#10 clk = !clk;

always@(posedge clk)

begin

if(data < 9'b11111111)

data = data + 1;

else

data = 0;

end

assign Cin = data[8];

assign A = data[7:4];

assign B = data[3:0];

endmodule